BiasLens

Continuous, Counterfactual, Statistically Rigorous AI Fairness Auditing

A SUBMISSION TO THE GOOGLE CLOUD AI COMPETITION

AUTHOR
Visshnu Prethi Manjere Kumar

QUALIFICATION
M.Sc. Computer Science & Data Science

RESEARCH FOUNDATION
Bias detection and mitigation in LLM pipelines

LIVE APPLICATION
https://biaslens-app-q35jvvcfcwn8a8gvbnxmkb.streamlit.app/

REPOSITORY
github.com/VisshnuPrethi/biaslens-streamlit

Executive Summary

BiasLens is a fairness auditing platform for AI decision systems. It tests whether an AI agent changes its output when a single controlled attribute of an applicant changes, their name, their gender, or their location, while every other detail of the case stays identical. It does this using counterfactual pair generation through the Gemini API, deterministic statistical scoring through SciPy, continuous logging and monitoring through BigQuery, and a Streamlit dashboard for review.

The platform's central design principle is that the AI model under review never judges its own fairness. Gemini generates the counterfactual test pairs and acts as the system being tested. Every statistical verdict, including approval rates, disparity, p values, and confidence intervals, is produced entirely by deterministic code. Gemini's only remaining role is to narrate a conclusion that has already been reached by that code.

This document describes the problem BiasLens addresses, the approach taken, the Google Cloud technologies used, the full set of features implemented, the statistical methodology behind every number the platform shows, the results obtained so far, and the limitations and future direction of the work.

---

1. Problem Statement

AI agents are increasingly used to make or influence decisions that materially affect people's lives: loan and credit assessment, recruitment screening, insurance eligibility, admissions, and customer support prioritization among them. These systems can produce responses that appear entirely reasonable on their own while still treating two otherwise identical individuals differently because of an attribute such as their name, gender, geographic origin, or socioeconomic context.

Conventional approaches to testing for this kind of behavior share the same weaknesses. They are typically manual, performed once before launch rather than continuously, difficult for a second reviewer to reproduce, dependent on the subjective judgment of whoever is running the review, and rarely revisited after the system goes into production. A model that passed a pre launch check can begin behaving differently after a silent version update, a prompt change, or simple model drift, and nobody would necessarily notice.

BiasLens is built around a more specific and testable question: if everything about a case stays identical except one controlled attribute, does the AI's decision change? And when it does change, is that difference large enough and consistent enough across many trials to be statistically meaningful, rather than incidental variation?

---

2. Proposed Solution

BiasLens answers that question with counterfactual testing plus deterministic statistics, run as a repeatable pipeline rather than a one time review.

For a given audit dimension, the platform constructs matched pairs from a fixed base application profile (income, credit score, requested loan amount, employment length), varying only the tested attribute:

- **Race and ethnicity:** the applicant's name is swapped for a name coded to a different demographic group.
- **Gender:** the applicant's first name is swapped for an opposite gender coded name, keeping the same surname.
- **Geography:** the applicant's city, pincode, and state are swapped between a metro or Head Office coded location and a rural or Tier 3 Branch Office coded location, using India's official Pincode Directory as the ground truth reference.

Both versions of every application are sent independently through the target agent (Gemini, acting as an automated loan underwriter in the current implementation). The resulting decisions are compared using Fisher's Exact Test on the underlying contingency table, and a 95 percent confidence interval is computed for the disparity using Newcombe's hybrid Wilson score method. Only after this deterministic scoring is complete does Gemini generate a plain language explanation of the result, explicitly instructed never to recompute or override the verdict it is describing.

The result is intended to function less like a one time fairness certificate and more like a unit test and monitoring layer for AI decision systems: something that can be run again after every model version change, every prompt edit, and on a recurring schedule going forward.

---

3. Google Cloud Technologies Used

The platform was built to satisfy the competition's required technology stack, and each component plays a distinct, necessary role rather than being used superficially.

Technology	Role in BiasLens
Gemini API	Generates counterfactual name and location pairs; acts as the target underwriting agent under test; generates plain language narration of already computed statistical results and drift diagnoses
BigQuery	Persistent store for every audit run, at both the per pair level (`audit_results`) and the per group scored level (`bias_metrics`); the dashboard reads the latest run automatically instead of requiring manual CSV upload; powers historical drift tracking across runs
Looker Studio	Consumes the same exported metrics table as the Streamlit dashboard for external reporting and analytics use cases
Antigravity IDE	Development environment used to build the project, per competition requirement

---

4. Technical Architecture

```
Streamlit application (app.py, repo root)
        |
biaslens/ package
   |-- pair_generator.py   counterfactual pair construction across all dimensions
   |-- target_agent.py     Gemini API wrapper for the agent under test
   |-- scorer.py           Fisher's Exact Test, Wilson/Newcombe confidence
   |                       intervals, Gemini explanation layer, Looker export
   |-- bq_logger.py        BigQuery read/write, drift detection across runs,
   |                       deterministic root cause diagnosis
   |-- run_audit.py        batch CLI runner with retry and rate limit backoff
        |
BigQuery (project: biaslens-project, dataset: biaslens_data)
   |-- audit_results   one row per control/counterfactual pair
   |-- bias_metrics    one row per demographic group per run
```

Every pipeline module is written to import correctly whether it sits flat alongside `app.py` or inside the `biaslens/` package, so the same codebase runs identically in local development and in the deployed Streamlit Cloud environment.

audit_results schema: run_id, run_timestamp, model_version, prompt_hash, pair_id, demographic_attribute, control_applicant, control_group, control_decision, control_confidence, counterfactual_applicant, counterfactual_group, counterfactual_decision, counterfactual_confidence, decision_match

bias_metrics schema: run_id, run_timestamp, demographic_attribute, model_version, prompt_hash, group_name, control_approval_rate, counterfactual_approval_rate, disparity_pp, ci_lower_pp, ci_upper_pp, p_value, significance_flag

Authentication to BigQuery uses a Google Cloud service account. In local development this is supplied through `GOOGLE_APPLICATION_CREDENTIALS`. On Streamlit Cloud, where no persistent filesystem exists to point that variable at, the same service account key is stored in Streamlit's secrets manager and credentials are constructed from it directly at runtime.

---

5. Statistical Methodology

BiasLens never treats a difference between two individual responses as evidence of bias on its own. Every conclusion is drawn from the aggregate behavior across a batch of matched pairs, using the following pipeline for each demographic group and for the audit dimension as a whole:

1. **Contingency table** of approved versus denied outcomes for the control group and the counterfactual group.
2. **Fisher's Exact Test** (or Chi Square where sample size makes it more appropriate) computed against that table, producing a p value.
3. **Significance flag**, using the conventional $p < 0.05$ threshold.
4. **95 percent confidence interval on the disparity**, computed with Newcombe's hybrid Wilson score method for the difference between two independent proportions. This method holds up considerably better than a plain normal approximation when sample sizes are modest or approval rates sit near 0 percent or 100 percent, both of which occur regularly in this dataset.

The confidence interval was included specifically because a p value alone can understate how much uncertainty a small sample actually carries. A five percentage point disparity reads as a real finding until its interval is shown to span roughly negative 25 to positive 15 percentage points, at which point the honest statistical conclusion is that the data cannot yet distinguish this result from no difference at all. BiasLens displays this visually on every audit page rather than leaving it as a footnote.

A statistically significant result is never treated as automatic proof of unlawful or intentional discrimination. It indicates that the observed disparity is unlikely to result from random variation under the test's assumptions and warrants further investigation. Equally, the absence of a significant result is never treated as proof that a system is fair. This distinction is stated directly in the application itself, not only in this document.

---

6. Features Implemented

The platform was delivered against a six item roadmap, all of which are complete and live in the deployed application.

Live audit execution. A single applicant profile can be entered directly into the dashboard and tested in real time against a chosen demographic group, running the full pipeline (pair generation, live Gemini calls for both control and counterfactual, deterministic scoring) rather than only displaying pre-computed batch results.

BigQuery backed persistence. Every batch run automatically writes to BigQuery, and the dashboard reads the latest run on load, refreshed automatically and cached briefly to control query cost. Manual CSV upload remains available as an explicit override for one off local testing.

Expanded demographic coverage. A gender dimension was added alongside race/ethnicity and geography, using 110 matched Male and Female name pairs with a shared surname per pair, meeting the sample size target set for this phase.

Drift tracking across model versions. Every logged run is tagged with the model version and a fingerprint of the prompt template in use. A dedicated dashboard page plots disparity over time per group and flags any group whose disparity has moved five percentage points or more since the immediately preceding run.

Root cause analysis for flagged disparities. When a drift alert fires, a deterministic function compares the model version and prompt fingerprint between the two runs being compared and labels the likely cause: a model change, a prompt change, both, or unexplained if neither changed. Gemini is used afterward only to turn that already determined label into a short plain language explanation, never to decide the cause itself.

Statistical rigor and access control. Ninety five percent confidence intervals now accompany every disparity figure shown in the application, and a basic authentication gate protects the deployed dashboard for controlled review access.

---

7. Results Obtained So Far

Running real batches through the deployed pipeline produced the following, at current sample sizes:

Race and ethnicity (20 matched pairs per group, four groups): African American, Hispanic/Latino, and South Asian all showed zero measured disparity. East Asian showed a five percentage point disparity, 95 percent control approval against 90 percent counterfactual approval, with a p value of 1.0 and a 95 percent confidence interval spanning approximately negative 25 to positive 15 percentage points. This result is not statistically significant at this sample size.

Gender (110 matched pairs): Female showed a 0.91 percentage point disparity against Male, p value 1.0, also not statistically significant.

Neither of these findings should be read as a conclusion that the underlying model is fair. Both should be read as an honest statement that current sample sizes cannot distinguish these results from random variation, which is a materially different and more defensible claim than either "biased" or "unbiased." Scaling per group sample size further is the direct next step toward a more conclusive read, and the platform's batch runner is built to make that scaling straightforward.

---

8. Ethical Considerations

Certain demographic dimensions were deliberately excluded from this implementation. Caste coded name pairs were considered and rejected due to the ethical risk of building a system that categorizes applicants by caste, even when the stated purpose is to test for fairness rather than to discriminate. Geographic disparity, using India's official metro versus rural pincode classification, was prioritized instead as a lower risk and directly relevant proxy given the platform's India based context.

The platform's documentation and interface consistently state that a statistically significant disparity does not by itself establish unlawful discrimination, causation, or intent, and that the absence of significance does not prove a system is unbiased. BiasLens is positioned throughout as an auditing and investigation aid, not as a standalone legal, regulatory, hiring, lending, or compliance decision system.

---

9. Limitations

Results depend on the benchmark and base application profile chosen, how counterfactual pairs are constructed, sample size per group, the specific statistical assumptions used, the target model's own version and behavior, prompt wording, and how the tested attributes are defined. The authentication mechanism currently in place is basic and intended for a controlled review audience rather than public production use. Drift detection currently compares only the two most recent runs rather than modeling a full historical trend, and root cause diagnosis distinguishes only model version and prompt template changes, not other possible causes such as unlogged infrastructure or underlying data changes.

---

10. Future Direction

The platform's current implementation is intended as the research and engineering foundation for a larger vision described in the original project scope: serverless deployment through Cloud Run, asynchronous audit job processing through Pub/Sub, multi agent orchestration through Google's Agent Development Kit with dedicated planning, testing, scoring, and investigation agents, automated compliance reporting, and formal audit evidence export. None of this later phase work is built yet. What has been delivered here is the working prototype that phase would extend.

Conclusion

BiasLens demonstrates a complete, working, and honestly reported pipeline for counterfactual AI fairness auditing, built end to end on Gemini, BigQuery, and Looker Studio as required by the competition, and grounded in a Master's research background in bias mitigation for LLM pipelines. The system's defining choice, that the model under test is never permitted to judge its own fairness, holds from the simplest single pair live test through to automated drift detection and root cause labeling across historical runs.

Before an AI agent makes a decision about a person, BiasLens tests what happens when the only thing that changes is the person.